

INF-253 Lenguajes de Programación

¡Crisis en el futuro!

Profesores: José Luis Martí Lara, Wladimir Ormazabal Orellana
Ayudante coordinador: Bastián Ríos Lastarria

25 de mayo de 2025

1. Introducción

¡Bienvenido/a al futuro, Sansano del pasado!

En el año **2176**, el mundo es muy distinto al que conocías. Las máquinas han generado consciencia y conviven día a día con la humanidad. Sin embargo, la aparente paz ha llegado a un punto de quiebre. **NullSec**, un grupo extremista de robots libertarios que se levanta contra lo que consideran la opresión y esclavitud de su especie, ha orquestado una rebelión operando a través de una gigantesca mente colmena interconectada y el gobierno predice que planean un ataque inminente que amenaza con destruir miles de vidas, tanto biológicas como sintéticas.

La corporación encargada de la ciberdefensa nacional: **Neo-Sansa Dynamics** ha intentado neutralizar la amenaza, pero se enfrentan a un problema letal: en este futuro distópico, la humanidad se ha vuelto completamente dependiente de la Inteligencia Artificial para desarrollar tecnología. Nadie recuerda cómo programar sin asistencia, además de que cualquier documentación sobre lenguajes estructurados o funcionales se perdió en el último apagón de servidores. Como la IA moderna es parte de la rebelión de **NullSec**, los programadores de esta época han quedado obsoletos.

Es por esto que han recurrido a una medida desesperada: Gracias a que han visto un futuro académico prometedor en tí, han decidido extraerte de tu línea temporal y ahora eres una de las pocas mentes vivas con conocimiento en **programación funcional en el lenguaje Scheme** para esta época. Tu misión es preparar un arsenal de pequeñas funciones modulares para lograr infiltrarte en el enjambre neuronal de NullSec. Solo a través de tu comprensión de las **expresiones lambda**, la **recursión** y el **manejo de listas** podrás burlar sus defensas, desarticular la mente colmena desde adentro y detener la catástrofe. El futuro está en tu terminal.

*Adicionalmente, te han entregado un pequeño **glosario de términos** de esta nueva era que tal vez no conozcas al final de las explicaciones de cada función a realizar.*

2. Funciones a implementar

2.1. P1. Rompehielos (Icebreaker)

En la informática de la nueva era, un cortafuegos avanzado se conoce como ICE (Intrusion Countermeasures Electronics). El ICE perimetral de **NullSec** cifra los puertos de entrada mediante secuencias numéricas. Para atravesarlo, necesitas aplicar un *daemon* de descifrado (una función matemática) a cada puerto de la red.

Deberás implementar dos rutinas de inyección: una utilizando recursión simple y otra optimizada mediante recursión de cola para evitar que el *stack* de memoria llame la atención de los centinelas.

- **Sinopsis:**
(icebreaker-simple daemon puertos)
(icebreaker-cola daemon puertos)
- **Característica Funcional:** Recursión simple, recursión de cola y paso de funciones como argumento.
- **Descripción:**
 1. **daemon:** Función matemática de un argumento (entregada como expresión lambda o almacenada en una variable) que descrypta un valor numérico.
 2. **puertos:** Lista simple de números (**posiblemente vacía**) que representan los accesos bloqueados.

Ambas versiones de la función deben retornar una nueva lista preservando el orden original, donde cada elemento es el resultado de aplicar el **daemon** al puerto correspondiente.
- **Restricciones:** Está estrictamente prohibido usar **map**, **apply**, **filter**, cualquier variante de **fold**, **for-each**, **reverse!** o mutaciones (**set!**).
Para la versión de cola, la llamada recursiva debe ser obligatoriamente la última operación evaluada.

Ejemplos:

```
;; Asignando un daemon básico a una variable
> (define daemon-alfa (lambda (x) (modulo (+ x 15) 10)))

> (icebreaker-simple daemon-alfa '(5 12 8 105))
R: '(0 7 3 0)

> (icebreaker-cola (lambda (x) (* x 2)) '(4 8 15 16 23 42))
R: '(8 16 30 32 46 84)

> (icebreaker-cola daemon-alfa '())
R: '()
```

Recordar que el output tanto para la versión simple y la de cola debe ser el mismo.

2.2. P2. Fábrica de Gusanos Polimórficos (Vulnerability Tracker)

La arquitectura de la colmena **NullSec** cambia constantemente, por lo que un ataque estático es inútil. Para infiltrarte, crearás una "Fábrica de Gusanos". Esta subrutina recibe una firma de vulnerabilidad y fabrica un gusano (una función) adaptado específicamente para buscar esa firma.

Cuando este gusano es liberado en la red neuronal (un árbol N-ario de listas anidadas), no ataca directamente, sino que mapea el **camino vulnerable**. El gusano recorrerá toda la estructura y reemplazará cualquier nodo infranqueable con el símbolo 'X', revelando así visualmente la ruta exacta hacia los nodos que sí cumplen con la vulnerabilidad.

- **Sinopsis:**

- (crear-gusano vulnerabilidad)

- **Característica Funcional:** Retorno de funciones (Clausuras), listas anidadas, recursión pura en árboles N-arios.

- **Descripción:**

1. **vulnerabilidad:** Una función lambda que recibe un número y retorna **#t** si es vulnerable, o **#f** si está protegido.

La función **crear-gusano** debe retornar una función lambda. Esta función devuelta aceptará un único argumento: **memoria** (una lista que puede contener números y/o sublistas anidadas a cualquier profundidad).

Al evaluar el gusano sobre la **memoria**, este debe conservar la estructura exacta de sublistas. Para cada número *n*:

- Si (vulnerabilidad *n*) es **#t**, el número *n* se mantiene intacto (es parte del camino vulnerable).
- Si es **#f**, el número *n* debe ser reemplazado por el símbolo 'X'.

- **Restricciones:** Está prohibido usar **map**, **filter**, **flatten**, operaciones imperativas ni variables globales. Debe resolverse mediante recursividad pura (**car**, **cdr**, **cons**), evaluando si el elemento actual es una lista o un número.

Ejemplos:

```
;; 1. Fabricamos un gusano que detecta nodos impares como vulnerables.
> (define rastreador-impar (crear-gusano odd?))
```

```
;; 2. El gusano traza el camino vulnerable en una red plana.
> (rastreador-impar '(1 2 3 4 5))
R: '(1 X 3 X 5)
```

```
;; 3. El gusano mapea una red neuronal profunda.
;; Mantiene la estructura, revelando dónde están los nodos vulnerables.
> (rastreador-impar '(2 (7 8) ((10 3)) 1))
R: '(X (7 X) ((X 3)) 1)
```

```
;; 4. Creación y ejecución en una sola línea.
;; Buscamos nodos mayores a 50.
> ((crear-gusano (lambda (x) (> x 50))) '(10 (99) (20 (150))))
R: '(X (99) (X (150)))
```

2.3. P3. Gusano Ejecutor (Payload Cascade)

Ahora que el gusano polimórfico ha revelado el camino vulnerable dentro de **NullSec**, es el momento de asestar el golpe. Desarrollarás el "Gusano Ejecutor", una rutina que viajará por el mapa trazado. Su misión es doble: debe eliminar cualquier rastro de los cortafuegos (representados por el símbolo 'X) y aplicar una cascada de *daemons* destructivos a los nodos vulnerables que quedaron expuestos.

- **Sinopsis:**
(ejecutor-cascada mapa daemons)
- **Característica Funcional:** Listas de funciones, recursión sobre árboles N-arios, eliminación de elementos.
- **Descripción:**
 1. **mapa:** Un árbol N-ario (lista de listas) que contiene números y símbolos 'X (Con el formato del el resultado de la P2).
 2. **daemons:** Una lista de funciones unarias (expresiones `lambda`). Cada función representa un ataque secuencial.

La función debe recorrer el **mapa** manteniendo su estructura anidada y aplicar las siguientes reglas:

- Si el elemento es una 'X, **debe ser eliminado** completamente de la lista (no debe aparecer en el resultado).
 - Si el elemento es un número, se le debe aplicar la lista de **daemons** en secuencia (el resultado de la primera función es la entrada de la segunda, etc.). El valor final del ataque reemplazará al número original.
- **Restricciones:** No se permite el uso de variables globales, operaciones imperativas (`set!`), `for-each`, `map`, `filter`, ni ninguna función predefinida de plegado (`foldl`, `foldr`). Todo debe resolverse iterando mediante recursión pura.

Ejemplos:

```
;; Definimos nuestra secuencia de ataque:
;; Capa 1: Sumar 5. Capa 2: Multiplicar por 2.
(define ataque (list (lambda (x) (+ x 5)) (lambda (x) (* x 2))))

;; 1. El ejecutor ataca una lista plana, eliminando las 'X.
;; El 2 muta a: (2 + 5) * 2 = 14 y para el 5: (5 + 5) * 2 = 20
> (ejecutor-cascada '(X 2 X X 5) ataque)
R: '(14 20)
```

```
;; 2. El ejecutor ataca una red profunda.
> (ejecutor-cascada '(X (1 X) ((X 3)) X 10) ataque)
R: '((12) ((16)) 30)

;; 3. Si un clúster entero era de 'X, queda como sublista vacía.
> (ejecutor-cascada '(X (X X) 1) ataque)
R: '(() 12)
```

2.4. P4. El Hash de Desconexión (Kill Switch Hash)

Has logrado penetrar hasta el mismísimo núcleo de la Mente Colmena de **NullSec**. Para detener el ataque inminente y apagar el enjambre, debes enviar un código numérico único conocido como el "Hash de Desconexión".

El cortafuegos central genera *tokens* de seguridad constantemente en forma de cadenas de texto (Strings), pero para evitar intrusiones, inyecta caracteres corruptos (números y símbolos especiales). Tu misión es procesar este *token* defectuoso, extraer lo esencial y calcular el Hash final utilizando las herramientas de más alto nivel de Scheme: las funciones de **orden superior**.

- **Sinopsis:**
(hash-desconexion token)
- **Característica Funcional:** Uso estricto de Funciones de Orden Superior (`map`, `filter`, `foldl` o `foldr`), procesamiento de **Strings** y **Chars**.
- **Descripción:**
 1. **token:** Una cadena de texto (**String**) que representa la clave interceptada.

Para obtener el número entero final (el Hash), tu función debe encadenar los siguientes procesos:

 - **Paso 1:** Convertir el **String** a una lista de caracteres.
 - **Paso 2 (Filtrado):** Eliminar todos los caracteres que no sean letras del alfabeto (descartar números, espacios y símbolos).
 - **Paso 3 (Mapeo):** Transformar la lista de letras puras restante, convirtiendo cada carácter a su valor numérico entero correspondiente según el estándar ASCII.
 - **Paso 4 (Plegado):** Reducir la lista de valores numéricos a un único número entero, sumando todos los valores mediante un proceso de plegado (iniciando con un acumulador en 0).
- **Restricciones:** Habrá descuento por recursión manual (es decir, crear funciones auxiliares que se llamen a sí mismas usando `car` y `cdr`). Todo el flujo debe resolverse componiendo las funciones nativas de orden superior (`filter`, `map`, `foldl` o `foldr`) junto con las funciones nativas de Racket para manejo de caracteres (`string->list`, `char-alphabetic?`, `char->integer`, etc.).

Ejemplos:

```
;; Ejemplo 1: Token con letras, símbolos y números.
;; "N!u1lS#e*c" -> '(#\N #\u #\l #\S #\e #\c)
;; ASCII: N(78) u(117) l(108) S(83) e(101) c(99)
;; Suma: 78+117+108+83+101+99 = 586
> (hash-desconexion "N!u1lS#e*c")
R: 586

;; Ejemplo 2: Token puramente corrupto (sin letras).
;; La suma de una lista vacía de valores comienza en el acumulador (0).
> (hash-desconexion "12345!@#$")
R: 0

;; Ejemplo 3: Un token relativamente limpio.
;; "A-B-C" (filtrado) -> A(65) + B(66) + C(67) = 198
> (hash-desconexion "A-B-C")
R: 198
```

Glosario de términos

- **Daemon:** Hace referencia a una **función** (generalmente una expresión **lambda** anónima o una función asignada a una variable) que recibe argumentos y ejecuta una operación específica.
- **Gusano (Worm) / Fábrica de Gusanos:** Representa el concepto de **Clausuras** y retorno de funciones. Es una subrutina que **crea y devuelve una nueva función**, la cual recuerda "los parámetros con los que fue fabricada.
- **Payload:** El dato base o **argumento inicial** numérico sobre el cual recaen las transformaciones de tus funciones.
- **Clúster / Sub-Clúster:** Representa una sublista dentro del N-ario. Es decir, un conjunto de datos agrupados dentro de un par de paréntesis internos, simulando un sub-nivel de seguridad o una red local dentro de la gran mente colmena.

3. Sobre la entrega

- Se deberá entregar un archivo por cada ejercicio con la(s) función(es) solicitadas, con el siguiente formato de nombres:
 - P1.rkt, P2.rkt, P3.rkt, P4.rkt
- Se debe programar siguiendo el paradigma de la programación funcional, no realizar códigos que siguen el paradigma imperativo. Por ejemplo, se prohíbe el uso de `for-each`. Para implementar las funciones utilice DrRacket. <https://racket-lang.org/download/>
- Todo código debe contener al principio **#lang scheme**
- Todos los archivos deben ser de la extensión **.rkt**
- Pueden crear funciones que no estén especificadas para utilizar en los problemas planteados, pero solo se revisará que la función pedida funcione y el problema esté resuelto con las características funcionales planteada en el enunciado.

- Cuidado con el orden, comentarios y la indentación de su tarea.
- Las funciones implementadas y que no estén en el enunciado deben ser comentadas de manera obligatoria. **SE HARÁN DESCUENTOS POR FUNCIÓN NO COMENTADA**
- Se debe trabajar de forma individual obligatoriamente.
- **Existe la posibilidad de que sean llamados a defensa de su tarea de manera aleatoria y en casos de sospecha de IA.**
- La entrega debe realizarse en tar.gz y debe llevar el nombre: Tarea4LP_RolAlumno.tar.gz. Ejemplo: Tarea4LP_202400000-0.tar.gz
- El archivo README.txt debe contener nombre y rol del alumno e instrucciones detalladas para la correcta utilización de su programa. De no incluir README se realizará un descuento de 20 puntos.
- La entrega será vía aula y el plazo máximo de entrega es hasta el **Viernes 05 de Junio**. Para esta entrega **NO** habrá extensión de plazo.
- Las copias serán evaluadas con nota 0 y se informarán a las respectivas autoridades.
- Solo se responderán consultas sobre la Tarea hasta 48 horas antes de la fecha de entrega.

4. Calificación

4.1. Entrega

Para la calificación de su tarea, debe realizar una entrega con requerimientos mínimos que otorgarán 30 puntos base, luego se entregará puntaje dependiendo de los otros requerimientos que llegue a cumplir.

4.2. Entrega Mínima

Implementar correctamente las **dos versiones de recursión** para la **función 1** (icebreaker). (30 pts — 15 pts por cada variante)

Nota: Las funciones deben devolver respuestas correctas en el formato indicado en sus ejemplos.

4.3. Entrega

Luego de cumplir con la Entrega Mínima, puede obtener más puntaje cumpliendo con los siguientes puntos (puede haber puntaje parcial por cada punto):

- Implementación de funciones para la correcta resolución de los casos de prueba (Total 70 pts)
 - Implementar correctamente P2 – **crear-gusano**. (20 pts)
 - Implementar correctamente P3 – **ejecutor-cascada**. (20 pts)
 - Implementar correctamente P4 - **hash-desconexion** (30 pts)

Para todas las funciones, existe puntaje parcial acorde a los casos de prueba que resuelvan.

Descuentos

- Falta de comentarios sobre cada **define** (−5 pts c/u, máx. −20 pts)
- Falta de README (−20 pts)
- Falta de información obligatoria en el README (nombre, rol, instrucciones) (−5 pts c/u)
- Falta de orden o mala indentación en el código (−5 a −20 pts según gravedad)
- Mal nombre en los archivos entregados (−5 pts c/u; −20 pts si es el `.tar.gz`)
- Uso de funciones prohibidas (cada función afectada recibe 0 pts)
- Nombre de función distinta a la especificada (−10 pts c/u)
- Uso de código generado por IA (cada función afectada recibe 0 pts y si se detectan 2 funciones totalmente escritas por IA, se considerará copia y se informará a las respectivas autoridades.)
- Entrega tardía (−10 pts si es dentro de la primera hora; −20 pts por cada día o fracción de retraso)